Projeto BDII PW APS

Profa. Ma. Edina Alves

Banco de Dados II + Programação para Internet + Análise e Projeto de Dados

Guia didático para desenvolvimento de um sistema web real

1. APRESENTAÇÃO DO PROJETO

Você e sua equipe deverão desenvolver um **sistema web funcional**, baseado em uma necessidade real de uma organização, empresa, comunidade ou serviço.

O projeto deverá integrar conhecimentos de três componentes curriculares:

- **Banco de Dados II** → modelagem, normalização, SQL, relacionamentos, constraints e implementação do banco;
- **Programação para Internet** → desenvolvimento da aplicação web em PHP, HTML e CSS, integração com banco e operações CRUD;
- **Análise e Projeto de Dados** → levantamento de requisitos, análise do problema, modelagem, documentação e organização do processo de desenvolvimento.

O desafio profissional

Imagine que sua equipe foi contratada por uma pequena empresa para desenvolver um sistema.

O cliente não quer apenas "uma tela bonita". Ele precisa de uma solução que:

1. resolva um problema real;
2. armazene informações de maneira organizada;
3. permita consultar, cadastrar, alterar e excluir dados;
4. reduza tarefas manuais;
5. seja fácil de utilizar;
6. tenha uma estrutura de banco de dados consistente;
7. possa futuramente receber novas funcionalidades.

O objetivo não é criar o sistema mais complexo. É demonstrar que vocês conseguem transformar um problema de negócio em uma solução tecnológica funcional.

2. TEMAS DISPONÍVEIS

Cada equipe deverá escolher **um dos temas**.

Os temas foram selecionados para apresentar dificuldade intermediária e permitir a integração das três disciplinas.

— Sistema de Agendamento de Serviços

Contexto

Pequenos negócios como **salões de beleza /barbearias, clínicas, oficinas, petshop** e prestadores de serviços frequentemente utilizam agendas em papel ou aplicativos genéricos.

Sistema proposto

Desenvolver uma aplicação para:

- cadastrar clientes;
- cadastrar profissionais;
- cadastrar serviços;
- registrar agendamentos;

- consultar agenda;
- alterar status do atendimento;
- cancelar agendamentos.

Possíveis entidades

- Cliente
- Profissional
- Serviço
- Agendamento
- Status

Relevância profissional

Permite trabalhar com **relacionamentos, datas, horários, regras de negócio e consultas**, situações muito comuns em sistemas comerciais.

3. REGRA FUNDAMENTAL DO PROJETO

Não comece programando.

O desenvolvimento deverá seguir esta sequência:

PROBLEMA → REQUISITOS → MODELO → BANCO → INTERFACE → PROGRAMAÇÃO → TESTES

Uma decisão tomada em uma fase deverá influenciar a próxima.

Por exemplo:

> Se o requisito diz que um cliente pode realizar vários agendamentos, isso deverá aparecer no modelo de dados.

E:

> Se existe uma entidade `Cliente` no modelo, deverá existir uma estrutura correspondente no banco e uma funcionalidade correspondente na aplicação.

Essa coerência será um dos principais critérios de avaliação.

---

4. METODOLOGIA DE DESENVOLVIMENTO

Para este projeto será utilizada a metodologia **Kanban**, por ser mais simples de compreender e aplicar em um projeto acadêmico de curta duração.

4.1 Quadro Kanban

Criem um quadro contendo, no mínimo:

Coluna	Significado
Backlog	Atividades ainda não iniciadas
A Fazer	Atividades selecionadas para execução
Em Desenvolvimento	Atividade atualmente sendo realizada
Em Teste	Atividade concluída aguardando validação
Concluído	Atividade validada

Cada funcionalidade deverá ser transformada em uma tarefa.

Exemplo

Cadastrar cliente

Critérios de aceitação:

- [ ] Tarefa Pendente

formulário possui nome;

- [ ] Tarefa Pendente

formulário possui telefone;

- [ ] Tarefa Pendente

formulário possui e-mail;

- [ ] Tarefa Pendente

dados são validados;

- [ ] Tarefa Pendente

dados são gravados no banco;

- [ ] Tarefa Pendente

sistema apresenta mensagem de sucesso;

- [ ] Tarefa Pendente

cliente aparece na listagem.

FASE 1 — PLANEJAMENTO E REQUISITOS

Objetivo

Compreender o problema antes de desenvolver a solução.

5.1 Defina o problema

Respondam:

1. Qual problema o sistema pretende resolver?
2. Quem possui esse problema?
3. Como o problema é resolvido atualmente?
4. Quais dificuldades existem no processo atual?
5. Como o sistema poderá melhorar esse processo?

Exemplo

Problema/Estudo de caso:

> Uma pequena assistência técnica controla suas ordens de serviço em planilhas e possui dificuldade para localizar rapidamente o histórico dos equipamentos.

Solução:

> Desenvolver um sistema web para registrar clientes, equipamentos, ordens de serviço e histórico de atendimentos.

5.2 Identifique os atores

Quem utilizará o sistema?

Exemplos:

- administrador;
- funcionário;
- técnico;
- cliente;
- professor;
- atendente.

Nem todo projeto precisa possuir vários tipos de usuários.

5.3 Levante os requisitos funcionais

Requisitos funcionais representam **o que o sistema deve fazer**.

Utilizem a seguinte estrutura:

> **RF01 — Cadastrar Cliente**
>
> O sistema deverá permitir ao usuário cadastrar um cliente informando nome, telefone e e-mail.

Exemplos:

- RF01 — Cadastrar cliente;
- RF02 — Alterar cliente;
- RF03 — Excluir cliente;
- RF04 — Consultar clientes;
- RF05 — Cadastrar produto;
- RF06 — Registrar movimentação.

5.4 Levante os requisitos não funcionais

Representam características ou restrições do sistema.

Exemplos:

RNF01 — Usabilidade

O sistema deverá apresentar interface simples e intuitiva.

RNF02 — Segurança

RNF03 — Compatibilidade

RNF04 — Desempenho

5.5 Regras de negócio

As regras de negócio são fundamentais.

Exemplo de sistema de biblioteca:

> RN01 — Um livro não poderá ser emprestado caso esteja indisponível.

Exemplo de sistema de eventos:

> RN02 — Não será possível realizar inscrição quando o número de participantes atingir o limite de vagas.

Exemplo de estoque:

> RN03 — Uma saída não poderá resultar em estoque negativo.

FASE 2 — MODELAGEM DE DADOS

Agora vocês transformarão os requisitos em uma estrutura de dados.

A pergunta central será:

> **Quais informações o sistema precisa armazenar e como elas se relacionam?**

6. MODELO CONCEITUAL — MER

Comecem identificando os **substantivos importantes** presentes nos requisitos.

Exemplo:

> "O cliente realiza agendamentos para serviços."

Possíveis entidades:

- Cliente;
- Agendamento;
- Serviço.

Depois identifiquem os relacionamentos:

> Cliente **realiza** Agendamento.

> Agendamento **refere-se a** Serviço.

6.1 Para cada entidade, identifique

- nome da entidade;
- atributos;
- identificador;
- relacionamentos;
- cardinalidade.

7. MODELO LÓGICO — DER

Depois do MER, transforme o modelo conceitual em tabelas.

Cada entidade normalmente dará origem a uma tabela.

8. NORMALIZAÇÃO ATÉ A 3FN

O banco deverá ser analisado e normalizado até a **Terceira Forma Normal (3FN)**.

1FN — Primeira Forma Normal

Cada campo deverá conter um valor atômico.

Evitem:

```
telefones = "11999999999 / 11888888888"
Endereço (composto)
```

Prefiram uma estrutura apropriada para múltiplos telefones, caso isso seja requisito do sistema.

2FN — Segunda Forma Normal

Além de estar na 1FN, os atributos devem depender da chave primária completa.

Esse problema aparece principalmente em tabelas com chaves compostas.

3FN — Terceira Forma Normal

Os atributos não devem depender de outros atributos que não sejam a chave.

Exemplo inadequado:

```
CLIENTE
id_cliente
nome
id_cidade
nome_cidade
```

Se `nome_cidade` depende de `id_cidade`, considere separar:

```
CIDADE
id_cidade
nome
```

e utilizar:

```
CLIENTE
id_cliente
nome
id_cidade
```

---

9. DICIONÁRIO DE DADOS

Para cada tabela, documentem os campos.

Exemplo:

Tabela	Campo	Tipo	Tamanho	PK	FK	Nulo	Descriçã
cliente	id_cliente	INT	—	Sim	Não	Não	Identifica
cliente	nome	VARCHAR	100	Não	Não	Não	Nome do cliente
cliente	email	VARCHAR	150	Não	Não	Sim	E-mail

O dicionário deverá ser coerente com o DER e posteriormente com o banco físico.

---

10. MODELO FÍSICO

Agora o modelo será transformado em SQL.

A equipe deverá produzir um script de criação da database.

FASE 3 — DESENVOLVIMENTO DA INTERFACE

Agora começa a construção da aplicação.

Tecnologias obrigatórias:

- **HTML**
- **CSS**
- **PHP**
- **Banco de dados relacional**

A aplicação deverá utilizar o banco desenvolvido na Fase 2.

---

11. ESTRUTURA MÍNIMA DO SISTEMA

O projeto deverá possuir, no mínimo:

Página inicial

Apresentar:

- nome do sistema;
- objetivo;
- menu de navegação;
- principais funcionalidades.

Página de cadastro

Formulário para inserção de dados.

Página de consulta

Listagem dos registros existentes.

Página de edição

Permitir alterar registros.

Exclusão

Permitir remover registros quando essa operação fizer sentido para o domínio.

---

12. CRUD

O sistema deverá possuir pelo menos um CRUD completo.

CRUD significa:

Operação	Significado
C	Create — criar
R	Read — consultar
U	Update — atualizar
D	Delete — excluir

Exemplo

Para clientes:

```
Cadastrar cliente
       ↓
Consultar clientes
       ↓
Editar cliente
       ↓
Excluir cliente
```

13. VALIDAÇÃO

O sistema deverá validar os dados.

Exemplos:

- campos obrigatórios;
- formato de e-mail;
- datas;
- valores numéricos;
- registros duplicados;
- relacionamentos existentes;

- regras de negócio.

Não confiem apenas na interface.

A aplicação também deverá possuir validação no lado do servidor.

14. ENTREGA FINAL DO PROJETO

A equipe deverá entregar um arquivo Word com toda documentação

Apresentação em canva e o Sistema funcionando ao vivo.